

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Data Interpretation

COURSE NAME: COMPUTER VISION COURSE CODE: DSA0204

COURSE OUTCOME COVERED

CO4: Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics. (BTL 5)

Assessment Tool Weightage: 40%

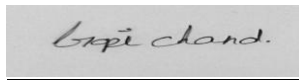
Read the given data carefully and compare the techniques or results based on multiple factors such as accuracy, robustness, and computational constraints. Evaluate and justify decisions with appropriate reasoning considering the application context.

Code of Conduct

I G. Gopi Chand (192424384) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. Real-Time System Design Trade-off

A surveillance system must operate under constraints: Viola-Jones (fast, 78% accurate), YOLO (moderate speed, 92% accurate), Deep Learning (95% accurate, slow).

Comparison: Viola-Jones offers the lowest latency but sacrifices accuracy (78%), making it unreliable for security-critical detection. Deep Learning delivers the highest accuracy (95%) but its slow inference makes it unsuitable where frames must be processed in real time. YOLO sits between the two, combining near-real-time speed with strong accuracy (92%).

Evaluation: For a surveillance system with strict latency constraints, missed detections due to speed (Viola-Jones) and missed frames due to lag (Deep Learning) are both operationally risky. YOLO's single-pass architecture keeps inference time low while its accuracy is close to that of the deep model.

Justification: YOLO is selected because it satisfies the hard latency constraint while keeping accuracy within an acceptable margin of the best-performing model, giving the best accuracy-per-millisecond trade-off for continuous real-time surveillance.

2. Robust Detection under Environmental Variations

Detection accuracy of Viola-Jones, YOLO and Deep Learning is measured under Normal, Occlusion and Low Light conditions (see table).

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%

Comparison: Viola-Jones degrades sharply under occlusion (60%) and low light (55%), showing weak robustness to environmental change. YOLO holds up better (75% occlusion, 70% low light) but still loses accuracy relative to normal conditions. Deep Learning shows the smallest drop across all conditions (85% occlusion, 80% low light), indicating superior feature robustness.

Evaluation: Deep Learning provides the best overall performance across varying conditions because its learned hierarchical features generalize better to occlusion and illumination changes than the hand-crafted Haar features of Viola-Jones or the single-shot grid features of YOLO.

Justification: Although Deep Learning is computationally heavier, its consistently higher accuracy across all environments makes it the most reliable choice when robustness across unpredictable real-world conditions is prioritized over raw speed; YOLO remains the pragmatic middle ground when some latency budget still exists.

3. Optical Flow Reliability vs Noise Sensitivity

Method A: stable vectors but misses small motions. Method B: detects fine motion but produces noisy vectors.

Comparison: Method A prioritizes stability, making it reliable for tracking large, coherent movements but blind to subtle motion cues. Method B captures fine-grained motion detail that A would miss, but its noise reduces vector reliability, which can generate false motion trails.

Evaluation: In dynamic scenes where small, fast movements matter (e.g., gesture or crowd analysis), Method B's sensitivity is valuable, but raw noisy vectors are difficult to use directly for tracking decisions without post-processing.

Justification: Method B is more suitable for dynamic scenes when paired with a smoothing/filtering stage (e.g., median or Kalman filtering) to suppress noise while retaining fine-motion sensitivity; used alone, Method A is preferable only when scene motion is coarse and stability is the priority.

4. Background Modeling Strategy (GMM)

A GMM-based system shows high false positives in dynamic backgrounds and missed detections in static backgrounds.

Comparison: In dynamic backgrounds (e.g., swaying trees, water), GMM's Gaussian components adapt too slowly to rapid pixel intensity changes, flagging background motion as foreground. In static backgrounds, GMM's adaptive learning rate can absorb slow-moving or low-contrast foreground objects into the background model, causing missed detections.

Evaluation: GMM alone is not sufficient across both scenario types because its single global learning-rate parameter cannot simultaneously satisfy the fast-adaptation needs of dynamic scenes and the sensitivity needs of static scenes.

Justification: Improvements should include an adaptive/per-pixel learning rate, shadow suppression, and morphological post-filtering to cut false positives, combined with a lower detection threshold or fusion with optical flow/frame-differencing to catch missed detections in static regions — a hybrid approach outperforms GMM in isolation.

5. Motion Estimation vs Computational Cost

Method A: low error (5%), high computation. Method B: moderate error (10%), low computation.

Comparison: Method A achieves twice the accuracy of Method B but at a significantly higher computational cost, while Method B trades some precision for speed and efficiency.

Evaluation: Real-time applications (e.g., live video tracking, robotics) are bounded by processing budgets per frame, so an accurate but slow method risks frame drops; offline applications (e.g., video post-production, forensic analysis) have no such deadline and can fully exploit accuracy.

Justification: Method B is preferable for real-time systems since its lower computation keeps up with frame rate despite the accuracy trade-off, while Method A is justified for offline processing where the 5% error reduction is worth the extra computation time.

6. Structure from Motion Consistency

Dataset A: accurate depth but requires high-quality images. Dataset B: inconsistent depth but works on low-quality images.

Comparison: Dataset A produces more reliable depth reconstruction but is constrained by strict image-quality requirements that are not always achievable in the field. Dataset B trades reconstruction consistency for tolerance to noisy, low-resolution, or poorly-lit captures.

Evaluation: Real-world deployment (e.g., outdoor mapping, consumer devices, mobile capture) rarely guarantees high-quality, controlled imaging conditions, so a method that fails outright on suboptimal input has limited practical value even if more accurate in ideal conditions.

Justification: Dataset B's setup is more practical for general real-world deployment because it degrades gracefully rather than failing on realistic input; Dataset A remains the better choice only in controlled environments (e.g., studio scanning, surveying) where high-quality capture can be guaranteed.

7. Stereo Vision vs Motion Estimation

Stereo Vision: accurate depth but requires dual cameras. Motion Estimation: less accurate but works with a single camera.

Comparison: Stereo vision exploits camera-pair disparity for precise, direct depth measurement, but its dual-camera requirement adds hardware cost, calibration overhead, and physical space constraints. Motion estimation infers depth cues from a single moving camera, sacrificing accuracy for hardware simplicity.

Evaluation: Under hardware-constrained deployments (e.g., low-cost embedded devices, drones, smartphones with a single lens), stereo vision is not physically viable, whereas motion estimation can still deliver usable, if less precise, depth information.

Justification: Motion estimation is preferable when hardware constraints rule out a dual-camera setup; stereo vision should be chosen whenever dual cameras are feasible, since it offers materially higher depth accuracy for applications like robotics or 3D mapping.

8. Model Generalization vs Overfitting

Model A: 98% training accuracy, 70% testing accuracy. Model B: 90% training accuracy, 88% testing accuracy.

Model	Training	Testing
A	98%	70%
B	90%	88%

Comparison: Model A shows a large gap (28 points) between training and testing accuracy, a classic sign of overfitting to the training data. Model B shows a much smaller gap (2 points), indicating it has learned generalizable patterns rather than memorizing training samples.

Evaluation: Deployment performance depends on how a model behaves on unseen data, not on training-set accuracy; Model A's high training score is misleading since it does not translate to real-world reliability.

Justification: Model B is more reliable for deployment because its small train-test gap indicates strong generalization and lower risk of unpredictable failures on new data, even though its peak accuracy is nominally lower than Model A's.

9. YOLO Performance across Scales

Detection results: Large objects 95%, Medium objects 85%, Small objects 60%.

Comparison: YOLO performs strongly on large objects but accuracy declines sharply as object size decreases, dropping 35 points from large to small objects.

Evaluation: The steep degradation on small objects reflects YOLO's coarse grid-cell resolution and downsampling, which causes small objects to occupy too few pixels/features to be reliably localized — this makes YOLO alone insufficient for multi-scale detection tasks.

Justification: Improvements such as Feature Pyramid Networks (FPN), multi-scale training, higher input resolution, anchor box tuning for small objects, or image tiling should be added; YOLO alone should not be relied upon where small-object detection (e.g., distant pedestrians, aerial imagery) is critical.

10. Motion Model Selection for Animation

Model A: smooth but physically inaccurate. Model B: realistic but computationally intensive.

Comparison: Model A produces visually smooth, low-cost motion at the expense of physical plausibility. Model B captures physically accurate motion but at a computational cost that can limit frame rate.

Evaluation: Real-time systems (e.g., interactive games) need consistent frame rates, so smoothness and low cost outweigh strict physical accuracy, whereas cinematic productions render offline and can afford to prioritize realism.

Justification: Model A should be used for real-time/interactive animation where speed and visual fluidity matter most, while Model B is justified for cinematic/offline rendering where computation time is not constrained and realism is the priority.